Exercício Prático: Banco de Dados Posto de Saúde

Modelagem Relacional (DDL), Povoamento de Tabelas (DML) e Consultas Avançadas em SQL

🎓 IDENTIFICAÇÃO ACADÊMICA & CONTEXTO DO CURSO

Estudante: Bruno Garcia — Graduando em **Ciência da Computação**

Formação Complementar: Curso de **Administração de Banco de Dados** pelo **IFRS** (Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul)

Ferramenta Utilizada: phpMyAdmin (Ambiente Web de Gerenciamento MySQL/MariaDB) • **Contato:** (17) 99651-9412

🌐 1. O Ambiente phpMyAdmin e a Gestão Online de Dados

Para o desenvolvimento e validação deste exercício prático, foi utilizado o **phpMyAdmin**, uma ferramenta de software livre escrita em PHP destinada à administração de bancos de dados **MySQL** e **MariaDB** diretamente pela Internet. Por funcionar de forma totalmente **online no navegador web**, o phpMyAdmin dispensa a instalação de clientes gráficos pesados na máquina local, oferecendo uma interface visual intuitiva acessível de qualquer navegador.

Através do painel web do phpMyAdmin, é possível criar e alterar estruturas de bancos de dados, gerenciar chaves primárias e estrangeiras, inserir e editar registros visualmente, além de executar diretamente scripts **SQL** complexos, criar *Views* e programar *Triggers* com retorno imediato de resultados e diagnósticos de execução.

📊 2. Arquitetura do Banco de Dados: Posto de Saúde (DDL)

O sistema modelado representa a operação clínica de um Posto de Saúde, contemplando o cadastro de pacientes, profissionais médicos, agendamentos de consultas, controle de medicamentos, prescrições médicas e solicitações de exames laboratoriais. Abaixo apresenta-se o resumo estrutural das 7 tabelas criadas:

Tabela	Chave Primária	Chaves Estrangeiras	Finalidade no Sistema
pacientes	`id` (AI)	—	Armazena nome, data de nascimento, CPF (único), telefone e endereço.
medicos	`id` (AI)	—	Registra nome, CRM (único), especialidade médica e telefone de contato.
consultas	`id` (AI)	`paciente_id`, `medico_id`	Vincula o paciente ao médico em uma data/hora específica com observações.

Tabela	Chave Primária	Chaves Estrangeiras	Finalidade no Sistema
medicamentos	`id` (AI)	—	Catálogo farmacêutico com princípio ativo, fabricante, lote e validade.
receitas_medicas	`id` (AI)	`consulta_id`, `medicamento_id`	Prescrições emitidas na consulta, definindo posologia e quantidade.
exames	`id` (AI)	—	Catálogo de exames disponíveis com nome e descrição clínica.
consultas_exames	(`consulta_id`, `exame_id`)	`consulta_id`, `exame_id`	Tabela associativa (N:N) para solicitações, datas e resultados de exames.

🔧 Script Completo de Criação das Tabelas (CREATE TABLE)

```
-- Tabela Pacientes
 pacientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    data_nascimento DATE NOT NULL,
    cpf VARCHAR(14) UNIQUE NOT NULL,
    telefone VARCHAR(15),
    endereco VARCHAR(255) NOT NULL
);


 medicos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    crm VARCHAR(15) UNIQUE NOT NULL,
    especialidade VARCHAR(100) NOT NULL,
    telefone VARCHAR(15)
);


 consultas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    data_hora DATETIME NOT NULL,
    paciente_id INT NOT NULL,
    medico_id INT NOT NULL,
    observacoes TEXT,
    FOREIGN KEY (paciente_id) REFERENCES pacientes (id),
    FOREIGN KEY (medico_id) REFERENCES medicos (id)
);


 medicamentos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    principio_ativo VARCHAR(100) NOT NULL,
    fabricante VARCHAR(100) NOT NULL,
    lote VARCHAR(20) NOT NULL,
    validade DATE NOT NULL
);


 receitas_medicas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    consulta_id INT NOT NULL,
    medicamento_id INT NOT NULL,
    posologia TEXT NOT NULL,
    quantidade INT NOT NULL,
    FOREIGN KEY (consulta_id) REFERENCES consultas (id),
    FOREIGN KEY (medicamento_id) REFERENCES medicamentos (id)
);
```

```
CREATE TABLE-- Tabela MédicosCREATE TABLE-- Tabela ConsultasCREATE TABLE-- Tabela MedicamentosCREATE TABLE-- Tabela Receitas MédicasCREATE TABLE-- Tabela ExamesCREATE TABLE-- Tabela Consultas_Exames (Relacionamento N:N)CREATE TABLE
```

📥 3. Povoamento Inicial das Tabelas (INSERT INTO)

Após a criação da estrutura relacional, as tabelas foram populadas com registros iniciais para permitir os testes de integridade referencial, junções e agregações no phpMyAdmin:

```
-- Inserir dados na tabela Pacientes
INSERT INTO pacientes (nome, data_nascimento, cpf, telefone, endereco) VALUES
('João Silva', '1985-07-10', '111.222.333-44', '(11) 91234-5678', 'Rua das Flores, 123, São Paulo'),
('Maria Souza', '1992-02-28', '222.333.444-55', '(11) 92345-6789', 'Rua das Pedras, 456, São Paulo');

-- Inserir dados na tabela Médicos
INSERT INTO medicos (nome, crm, especialidade, telefone) VALUES
('Dr. Pedro Ferreira', '12345-SP', 'Clínico Geral', '(11) 93456-7890'),
('Dra. Ana Oliveira', '67890-SP', 'Pediatra', '(11) 94567-8901');

-- Inserir dados na tabela Consultas
INSERT INTO consultas (data_hora, paciente_id, medico_id, observacoes) VALUES
('2023-03-15 09:00:00', 1, 1, 'Paciente com dores de cabeça frequentes.'),
('2023-03-15 10:00:00', 2, 2, 'Consulta de rotina para criança.');

-- Inserir dados na tabela Medicamentos
INSERT INTO medicamentos (nome, principio_ativo, fabricante, lote, validade)
VALUES
('Dorflex', 'Dipirona Monoidratada + Cafeína + Orfenadrina', 'Sanofi', 'A123B456', '2023-12-31'),
('Paracetamol', 'Paracetamol', 'Genéricos', 'X789Y1011', '2024-06-30');

-- Inserir dados na tabela Receitas Médicas
INSERT INTO receitas_medicas (consulta_id, medicamento_id, posologia, quantidade)
VALUES
(1, 1, 'Tomar 1 comprimido a cada 6 horas em caso de dor.', 10),
(1, 2, 'Tomar 1 comprimido a cada 8 horas em caso de febre.', 10);

-- Inserir dados na tabela Exames
INSERT INTO exames (nome, descricao) VALUES
('Hemograma completo', 'Exame de sangue que avalia as células sanguíneas.'),
('Glicemia em jejum', 'Exame de sangue que mede a taxa de glicose na corrente sanguínea.');

-- Inserir dados na tabela Consultas_Exames
INSERT INTO consultas_exames (consulta_id, exame_id, data_solicitacao, data_resultado, resultado) VALUES
(1, 1, '2023-03-15', '2023-03-17', 'Resultados normais.'),
(1, 2, '2023-03-15', '2023-03-17', 'Glicemia: 95 mg/dL.');
```

🔍 4. Resolução Comentada do Questionário SQL (20 Questões)

Abaixo estão documentados todos os comandos SQL desenvolvidos e validados para responder às 20 questões práticas do exercício sobre consultas, atualizações, exclusões, *Views* e *Triggers*:

1. Quantidade total de exames realizados em cada consulta, identificado pelo ID da consulta:

```sql
SELECT consultas.id, COUNT(consultas_exames.exame_id) AS total_exames
FROM consultas
JOIN consultas_exames ON consultas.id = consultas_exames.consulta_id
GROUP BY consultas.id;
```

2. Quantidade total de consultas realizadas por cada médico, identificado pelo nome:

```sql
SELECT medicos.nome, COUNT(consultas.id) AS total_consultas
FROM medicos
JOIN consultas ON medicos.id = consultas.medico_id
GROUP BY medicos.nome;
```

3. Atualizar o telefone de um paciente específico, utilizando o ID do paciente como filtro:

```sql
UPDATE pacientes
SET telefone = 'NOVO_TELEFONE'
WHERE id = ID_DO_PACIENTE;
```

4. Excluir um exame específico, utilizando o ID do exame como filtro:

```sql
DELETE FROM exames
WHERE id = ID_DO_EXAME;
```

5. Excluir a prescrição de um medicamento específico em uma consulta específica (ID consulta + ID medicamento):

```sql
DELETE FROM consultas_medicamentos
WHERE consulta_id = ID_DA_CONSULTA AND medicamento_id = ID_DO_MEDICAMENTO;
```

6. Atualizar o endereço de um paciente específico, utilizando o CPF do paciente como filtro:

```
UPDATE pacientes
SET endereco = 'NOVO_ENDERECO'
WHERE cpf = 'CPF_DO_PACIENTE';
```

7. Listar todos os pacientes que não possuem consultas registradas:

```
SELECT * FROM pacientes
WHERE id NOT IN (SELECT paciente_id FROM consultas);
```

8. Comando UPDATE para aumentar o preço de todos os exames em 10%:

```
UPDATE exames SET preco = preco * 1.1;
```

9. Criar um TRIGGER que atualiza a coluna 'ultima_consulta' da tabela pacientes após nova consulta:

```
CREATE TRIGGER atualiza_ultima_consulta
AFTER INSERT ON consultas FOR EACH ROW
UPDATE pacientes SET ultima_consulta = NEW.data_consulta WHERE id =
NEW.paciente_id;
```

10. Atualizar o nome de um medicamento específico, utilizando o ID do medicamento como filtro:

```
UPDATE medicamentos SET nome = 'NOVO_NOME'
WHERE id = ID_DO_MEDICAMENTO;
```

11. Comando UPDATE para alterar o CRM de um médico específico, utilizando o ID do médico como filtro:

```
UPDATE medicos SET crm = 'NOVO_CRM' WHERE id = ID_DO_MEDICO;
```

12. Listar todas as consultas de um médico específico (por ID), ordenadas pela data e hora da consulta:

```
SELECT * FROM consultas
WHERE medico_id = ID_DO_MEDICO
ORDER BY data_hora;
```

13. Listar todos os pacientes que nasceram antes de 1º de janeiro de 1990:

```
SELECT * FROM pacientes
WHERE data_nascimento < '1990-01-01';
```

14. Excluir um paciente específico, utilizando o ID do paciente como filtro:

```
DELETE FROM pacientes WHERE id = ID_DO_PACIENTE;
```

15. Criar uma VIEW que mostre o nome do paciente, nome do médico e data da consulta para todas as consultas:

```
CREATE VIEW consultas_detalhes AS
SELECT pacientes.nome AS paciente_nome, medicos.nome AS medico_nome,
consultas.data_consulta
FROM consultas
JOIN pacientes ON consultas.paciente_id = pacientes.id
JOIN medicos ON consultas.medico_id = medicos.id;
```

16. Remover todos os exames realizados em uma consulta específica, utilizando o ID da consulta como filtro:

```
DELETE FROM consultas_exames
WHERE consulta_id = ID_DA_CONSULTA;
```

17. Excluir um medicamento específico, utilizando o ID do medicamento como filtro:

```
DELETE FROM medicamentos
WHERE id = ID_DO_MEDICAMENTO;
```

18. Excluir uma consulta específica, utilizando o ID da consulta como filtro:

```sql
DELETE FROM consultas
WHERE id = ID_DA_CONSULTA;
```

19. Mostrar a quantidade total de consultas realizadas por cada médico, identificado pelo nome:

```sql
SELECT medicos.nome, COUNT(consultas.id) AS total_consultas
FROM medicos JOIN consultas ON medicos.id = consultas.medico_id
GROUP BY medicos.nome;
```

20. Criar um TRIGGER que impede a exclusão de um médico se ele tiver consultas agendadas:

```sql
CREATE TRIGGER impede_exclusao_medico
BEFORE DELETE ON medicos FOR EACH ROW
BEGIN
    IF (SELECT COUNT(*) FROM consultas WHERE medico_id = OLD.id) > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Não é possível excluir um
médico com consultas agendadas';
    END IF;
END;
```


👨‍💻 Informações do Autor & Repositório

👤 **Nome Completo:** Bruno Garcia

🎓 **Formação:** Estudante de Ciência da Computação • Curso de Administração de Banco de Dados (IFRS)

📱 **Celular / WhatsApp:** (17) 99651-9412

✉ **E-mail:** bruno.garcia.comp@gmail.com

💻 **GitHub:** github.com/BrunoGarciaCs/curso_bd_ifrs_exercicios